FestivalGo: A Full-Stack Web Platform for Personalized Music Festival Experiences and Social Connectivity

Ciceu Robert
Faculty of Automation and Computer Science
Technical University of Cluj-Napoca
Cluj-Napoca, Romania
ciceu.ma.robert@student.utcluj.ro

***Abstract*—The live events industry has evolved significantly, yet most digital platforms remain limited to ticketing services, neglecting the holistic user experience. This paper presents FestivalGo, a full-stack web application designed to bridge the gap between logistic organization and social interaction. Unlike traditional platforms, FestivalGo integrates a personalized recommendation system based on the "Big Five" psychological model and music preferences, along with interactive maps for facility location. The system is built using a layered architecture featuring a Spring Boot backend, a Vue.js frontend, and a MySQL relational database. Key contributions include a custom recommendation algorithm, real-time social group formation, and seamless integration with Stripe for payments and Leaflet for geospatial visualization.**

***Index Terms*—Web Application, Spring Boot, Vue.js, Recommender System, Music Festivals, Interactive Maps, Social Computing.**

I. Introduction

The live events industry has rapidly expanded, transforming music festivals from simple concerts into complex cultural phenomena [1]. In an era where technology shapes communication, festivals remain relevant by offering unique, shared experiences. However, the digital tools accompanying these events have not kept pace with user expectations.

Modern attendees desire more than just a schedule or a ticket; they seek personalized recommendations, interactive navigation, and social connectivity. While platforms like Eventbrite or Ticketmaster excel in sales, they lack features that facilitate the social and experiential aspects of a festival [3]. Furthermore, the fragmentation of services—users needing one app for tickets, another for maps, and a third for chatting—creates friction in the user journey.

This paper proposes **FestivalGo**, a unified web platform that centralizes the festival experience. The main objectives of this work are:

- **User-Centric Design:** Developing a full-stack application that prioritizes user experience (UX) over simple transactional utility.
- **Psychological Profiling:** Implementing a recommendation algorithm based on psychological traits and music preferences to suggest relevant events.
- **Geospatial Integration:** Integrating interactive maps for real-time orientation (stages, food courts, camping) using Leaflet.
- **Social Cohesion:** Facilitating social interaction through group chats and shared planning.

The rest of this paper is organized as follows: Section II analyzes related work and the theoretical background. Section III describes the system architecture and technology stack. Section IV details the implementation of key modules. Section V presents the results, and Section VI concludes the paper.

II. Related Work

To contextualize the development of FestivalGo, we performed a comparative analysis of existing platforms and reviewed relevant scientific literature regarding music psychology.

A. Comparative Analysis of Existing Platforms

Major platforms such as Eventbrite and Ticketmaster dominate the market. Eventbrite is renowned for its social media integration but lacks advanced interactive elements like live internal maps [2]. Ticketmaster provides robust ticketing solutions but does not offer social interaction features or personalized itinerary planning [3]. Locally, platforms like IaBilet.ro serve the Romanian market effectively for sales but do not include recommendation engines or community-building tools [4]. Table I highlights the functional gap that FestivalGo aims to fill.

TABLE I
Comparison with Existing Platforms

Feature	Eventbrite	Ticketmaster	IaBilet	FestivalGo
Ticketing	Yes	Yes	Yes	**Yes**
Social Groups	Limited	No	No	**Yes**
Interactive Maps	No	Static	No	**Dynamic**
Psych. Recomm.	No	AI (Basic)	No	**Big Five**

B. Theoretical Foundation: Music and Personality

The design of the FestivalGo recommendation system is grounded in the research of Rentfrow and Gosling [6]. Their study on the structure of music preferences suggests that musical taste is not random but correlated with personality traits (the ”Big Five” model).

They categorize music into four dimensions:

1) *Reflective & Complex* (Jazz, Blues, Classical).
2) *Intense & Rebellious* (Rock, Alternative, Metal).
3) *Upbeat & Conventional* (Pop, Country).
4) *Energetic & Rhythmic* (Electronic, Hip-Hop).

This theoretical framework supports the implementation of the application's quiz module. By mapping user answers to these dimensions, FestivalGo can generate recommendations that resonate on a deeper psychological level than simple genre tagging [7].

III. System Architecture

FestivalGo follows a client-server architecture, ensuring a clear separation of concerns (SoC) [9]. The system is divided into three main layers: the Presentation Layer (Frontend), the Business Logic Layer (Backend), and the Data Persistence Layer.

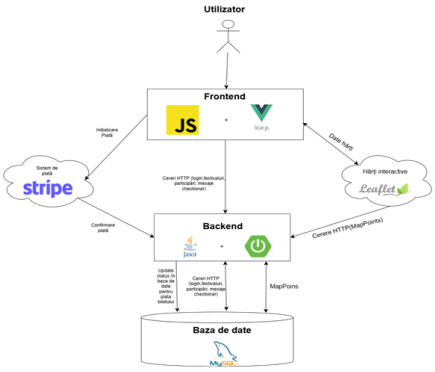

Fig. 1. General Schema

A. Backend Technology

The server-side is implemented in Java using the Spring Boot framework. Spring Boot was chosen for its ability to auto-configure dependencies and its embedded Tomcat server [8].

1) Security and Authentication: Security is a critical component. User authentication is handled via Spring Security.

- Password Encryption: Passwords are encrypted using the Bcrypt algorithm. Bcrypt is an adaptive one-way function that uses a work factor to resist brute-force attacks [11].
- Rate Limiting: To prevent abuse and Denial of Service (DoS) attempts on the login endpoint, we implemented Bucket4j [12]. This token-bucket algorithm limits the number of requests a user can make within a specific timeframe.

2) Data Access Layer: Interaction with the database is managed through Spring Data JPA, which abstracts standard CRUD operations. The use of JPA Repositories prevents SQL Injection attacks by using parameterized queries [10].

B. Frontend Technology

The user interface is built with Vue.js (v3), a progressive JavaScript framework selected for its component-based architecture.

- Build Tool: Vite is used for its superior performance compared to Webpack, offering instant server start and hot module replacement (HMR) [13].
- State Management: LocalStorage is utilized to persist non-sensitive session data (user ID, role) to maintain state across page reloads.
- Mapping:Leaflet.js is integrated for rendering interactive maps using OpenStreetMap data tiles [14].

C. Database Design

Data persistence is achieved using MySQL, a relational database management system. The choice of a relational database over NoSQL was driven by the need for strong data consistency and complex relationships between entities.

Key entities include:

- *User*: Stores credentials and role (USER/ADMIN).
- *Festival*: Contains metadata (dates, location, description).
- *MapPoint*: Stores geospatial coordinates (latitude, longitude), linked to a specific Festival via a Many-to-One relationship.
- *Quiz/Answer*: Stores the psychological profiling data.

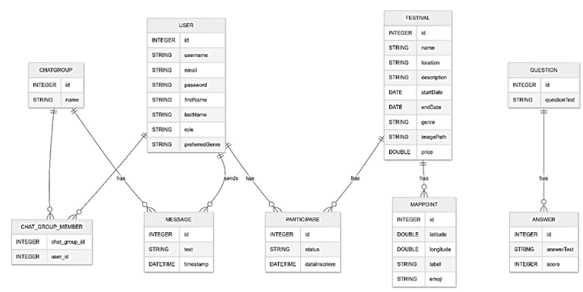

Fig. 2. Entity-Relationship Diagram (Simplified).

IV. Implementation Details

This section details the logic behind the core functionalities that differentiate FestivalGo from standard ticketing apps.

A. The Recommendation Module (The Quiz)

The recommendation engine is the unique selling point of the platform. It consists of a 21-question quiz designed to map the user's personality to the four musical dimensions defined by Rentfrow and Gosling.

The algorithm functions as follows:

1) The user answers questions.

2) Each answer is associated with a specific weight affecting a musical dimension (e.g., ”Do you enjoy complex puzzles?” might increase the score for *Reflective & Complex*).
3) Upon completion, the backend aggregates the scores.
4) The system queries the ‘Festival‘ table to find events tagged with genres matching the highest-scoring dimension.
5) A list of recommended festivals is returned to the frontend.

B. Interactive Geospatial Module

The spatial orientation component is implemented using the **Leaflet.js** library, integrated within the Vue.js component lifecycle. The map initialization occurs during the mounting phase (‘onMounted‘), attaching the interactive layer to a specific DOM element centered on the festival’s coordinates.

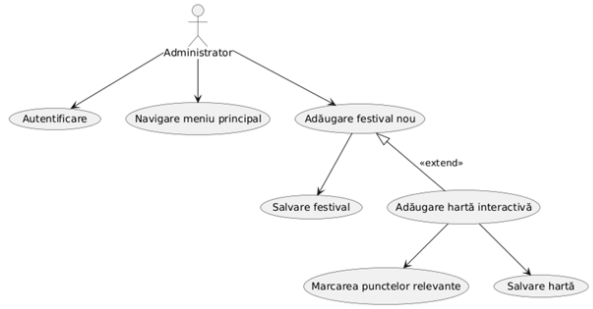

Fig. 3. Use Case Diagram for creating a festival and adding the associated map.

1) Data Modeling and Persistence: On the backend, a dedicated package manages geospatial data. The core entity, *MapPoint*, is defined with attributes for unique identification, latitude, longitude, and visual metadata (emoji type and label). The data access layer is handled via a Spring Data repository, while a specific Service class implements the business logic for creating and updating points.

2) Admin Interaction and Marker Customization: The administration interface allows for dynamic point placement. Upon a map click event, the system captures the geographic coordinates and prompts the administrator to assign a specific emoji. Visual representation is achieved using Leaflet’s *divIcon* class, which renders the emoji and its associated label as a custom marker. These points are temporarily stored in a frontend list state.

This allows users to zoom, pan, and click on markers to see details about stages or restrooms.

C. Payment Integration via Stripe

To enable secure ticketing, the platform integrates the **Stripe** payment processor. The implementation prioritizes security by delegating sensitive data handling to Stripe’s secure environment.

1) Backend Configuration and Session Management: The integration begins with the secure storage of API credentials. The *Stripe Secret Key* is retrieved from the official dashboard and injected into the server’s `application.properties` configuration file, ensuring it is never exposed to the client side. A dedicated service within the Participation package manages the checkout flow. Upon receiving a purchase request, this service:

1) Authenticates with Stripe using the stored credentials.
2) Generates a *Checkout Session* object containing transaction details (amount, currency, festival name).
3) Returns the Session ID and the redirection URL to the frontend.

2) Client-Side Execution: The frontend utilizes the `loadStripe` utility to initialize the payment interface. When the user triggers the ”Buy Ticket” action:

- The system retrieves the User ID and Festival ID from *localStorage*.
- A POST request is sent to the backend to initiate the session.
- Essential context data is temporarily cached in local storage to maintain state during the external redirection.
- The user is redirected to the hosted Stripe Checkout page to complete the transaction securely.

3) Transaction Finalization: Upon a successful transaction, Stripe redirects the user back to the application’s specific route handled by the *PaymentSuccess.vue* component. This component triggers the final confirmation logic:

1) It intercepts the return signal.
2) Sends a confirmation POST request to the backend.
3) The *ParticipationService* updates the ticket status in the MySQL database from *PENDING* to *PAID*.
4) Finally, the user is redirected to their ”My Tickets” dashboard.

D. Social Connectivity and Group Messaging

To facilitate interaction between attendees, the platform includes a dedicated chat system. The implementation follows the standard layered architecture, divided into data modeling, business logic, and API exposure.

1) Data Modeling: The module relies on two primary entities: *ChatGroup* and *Message*.

- **ChatGroup:** Represents a conversation room, defined by a unique ID and a name. It maintains a *Many-to-Many* relationship with the *User* entity, allowing a single group to contain multiple members and a user to participate in multiple groups simultaneously.
- **Message:** Stores the communication content. It includes a unique ID, the message text, a timestamp, and a reference to the sender, modeled as a *Many-to-One* relationship with the *User* entity.

Data access is handled via Repository interfaces extending *JpaRepository*, providing standard CRUD capabilities without the need for custom query implementation.

2) Business Logic Services: The core logic is encapsulated in two service classes:

- **ChatGroupService:** Manages group lifecycle. The creation method validates the initiating user via *UserRepository*, instantiates a new group, automatically adds the creator as the first member, and persists the entity. Similarly, the member addition logic verifies the existence of both the group and the target user before updating the association table.
- **MessageService:** Handles data transmission. Before persisting a message, the system performs a security check to ensure the sender is an active member of the target group. Message retrieval involves querying the database for all records associated with a specific group ID, ordered chronologically by the timestamp.

3) REST API Layer: The interaction is managed by the *ChatGroupController*, mapped to the `/chat/groups` route. The controller exposes several key endpoints:

- `POST /create`: Instantiates a new group.
- `POST /{groupId}/add`: Adds a user to an existing group.
- `POST /{groupId}/message`: Persists a new message after validation.
- `GET /{groupId}/messages`: Retrieves the conversation history.

V. Results and Conclusions

A. Performance Evaluation

The application's performance was assessed by measuring the response time (latency) of the critical RESTful endpoints. The evaluation was conducted using **Postman** for API isolation testing and browser-based development tools to simulate the real-user experience. The analysis focused on standard HTTP operations (GET, POST, PUT, DELETE) representative of a typical user journey.

Table II summarizes the latency results for key operations. The recorded times indicate a highly responsive system, with most operations completing in under 400 milliseconds.

TABLE II
API Response Times for Critical Operations

Operation	HTTP Method	Latency (ms)
User Authentication (Login)	POST	95 – 429
User Registration	POST	394
Fetch All Festivals	GET	329
Fetch Quiz Questions	GET	120
Delete Festival (Admin)	DELETE	180

Furthermore, the system configuration was optimized for media handling. To ensure high-quality visual content for festival presentations, the multipart file upload limit was increased to **10 MB**. This configuration prevents *MaxUploadSizeExceeded* exceptions while maintaining efficient memory management during image processing.

B. User Experience

The integration of the "Big Five" based recommendations offers a significant improvement over generic "most popular" lists found on other platforms. The interactive map reduces anxiety related to venue navigation, addressing the "Spatial Authenticity" concept discussed by Skandalis et al. [5].

C. Conclusion

FestivalGo successfully demonstrates that a music festival platform can go beyond simple ticketing. By combining modern web technologies (Spring Boot, Vue.js) with psychological theories of music preference, the platform offers a holistic experience.

D. Future Work

Future development directions include:

- Mobile App: Developing a native mobile application to leverage GPS for real-time on-site tracking.
- Real-Time Notifications: Implementing WebSockets for instant chat updates.
- AI Integration: Enhancing the recommendation algorithm with Machine Learning based on user history.

References

[1] M. Bramhe, et al., "Online Event Management System: A critical Review," *Int. J. of Innovations in Eng. and Science*, vol. 9, no. 5, pp. 11-13, 2024.
[2] Eventbrite, "Platform Features," [Online]. Available: https://www.eventbrite.com.
[3] Ticketmaster, "Concerts and Tours." [Online]. Available: https://www.ticketmaster.com.
[4] IaBilet.ro, "Evenimente și bilete online." [Online]. Available: https://www.iabilet.ro.
[5] A. Skandalis, E. Banister and J. Byrom, "Spatial Authenticity and Extraordinary Experiences," *Journal of Travel Research*, vol. 63, no. 2, pp. 357–370, 2024.
[6] P. J. Rentfrow and S. D. Gosling, "The Do Re Mi's of Everyday Life: The Structure and Personality Correlates of Music Preferences," *Journal of Personality and Social Psychology*, vol. 84, no. 6, pp. 1236–1256, 2003.
[7] L. R. Goldberg et al., "The music of our lives: The role of musical preferences in predicting romantic attraction," *Psychology of Music*, vol. 44, no. 3, 2016.
[8] Spring, "Spring Boot Reference Documentation," 2024.
[9] T. Nguyen, "Reasons why Separation of Concerns (SoC) is essential," Medium, 2021.
[10] Spring, "Spring Data JPA Reference Documentation," 2024.
[11] Spring Security, "Reference Documentation," 2025.
[12] V. Kalashnikov, "Bucket4j – Java rate-limiting library," Bucket4j Documentation.
[13] Vite Team, "Why Vite?," Vite Official Guide, 2024.
[14] LeafletJS, "Leaflet: an open-source Javascript library for interactive maps," 2024.
[15] K. Harsh, "A Guide to Stripe Integration in Spring Boot Application," Kinsta Blog, 2025.
[16] Oracle Corporation, "MySQL 8.4 Reference Manual," 2024.